

Gitlab 搭建 私有代码仓库

广州威克斯软件科技有限公司

<https://gzcorvax.github.io>

2026 年 2 月

目录

Gitlab 搭建.....	1
私有代码仓库	1
目录	2
1.1 修改历史.....	1
1.2 所需软件.....	1
1.3 安装部署.....	1
1.3.1 硬件要求	1
1.3.2 安装方式	1
1.3.2.1 方式一	1
1.3.2.2 方式二	1
1.3.2.3 方式三	2
1.3.2.4 方式四	3
1.3.3 固定 IP	3
1.3.4 Gtlab 配置.....	5
1.3.5 用户密码	8
1.3.6 安装 Runner	10
1.3.7 常用命令	11
1.3.8 克隆项目	11
1.4 使用说明.....	16
1.4.1 创建 Group.....	16
1.4.2 创建 User	17
1.4.3 添加 User	18
1.4.4 创建 Project.....	19
1.4.5 添加成员	20
1.4.6 推送文件	21
1.4.7 中文设置	22
1.4.8 调整路径	23
1.4.9 禁止流水线	26
1.4.10 创建和注册 Runner	27
1.5 备份还原.....	31
1.5.1 备份 gitlab	31
1.5.2 恢复 gitlab	33
1.5.3 批量导出项目	34
1.5.4 导出项目	35
1.5.5 导入项目	37
1.6 参考文件.....	40

1.1 修改历史

序号	修改日期	修改人员	修改摘要
1	2026-02-12	考威克斯	创建

1.2 所需软件

- | | |
|--|---------------------------|
| 1. gitlab-ce_18.8.3-ce.0_amd64.deb | // gitlab 18 (ubuntu 安装包) |
| 2. gitlab-ce-18.8.3-ce.0.el10.x86_64.rpm | // gitlab 18 (centos 安装包) |
| 3. NetSarangXmanagerEnterprise5 | // Xmanager (远程连接工具) |

1.3 安装部署

1.3.1 硬件要求

Gitlab 要求服务器内存 2G 以上

1.3.2 安装方式

1.3.2.1 方式一

下载 gitlab-ce 的 rpm 包

- gitlab 官方 rpm 包下载
- 清华的源

将对应版本的 gitlab-ce 下载到本地后，直接 yum 安装即可

要先将这个 rpm 包下载到本地

```
yum install gitlab-ce-14.10.2-ce.0.el7.x86_64.rpm
```

1.3.2.2 方式二

地址：广州市 增城区 广深大道

电邮：gzcovax@126.com

邮编：510340

QQ：3987797287

配置 yum 源

在 /etc/yum.repos.d/ 下新建 gitlab-ce.repo, 写入如下内容:

```
[gitlab-ce]
name=gitlab-ce
baseurl=https://mirrors.tuna.tsinghua.edu.cn/gitlab-ce/yum/el7/
Repo_gpgcheck=0
Enabled=1
Gpgkey=https://packages.gitlab.com/gpg.key
```

然后创建 cache, 再直接安装 gitlab-ce

```
yum makecache # 这一步会创建大量的数据
```

```
# 直接安装最新版
```

```
yum install -y gitlab-ce
```

```
# 如果要安装指定的版本, 在后面填上版本号即可
```

```
yum install -y gitlab-ce-13.6.1
```

```
# 如果安装时出现 gpgkey 验证错误, 只需在安装时明确指明不进行 gpgkey 验证
```

```
yum install gitlab-ce -y --nogpgcheck
```

1.3.2.3 方式三

方式三: CURL

RedHat 环境

```
curl -s https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.rpm.sh | sudo bash
```

Ubuntu 环境

```
curl -s https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh | sudo bash
```

```
apt-get install gitlab-ce
```

地址: 广州市 增城区 广深大道

邮编: 510340

电邮: gzcovax@126.com

QQ: 3987797287

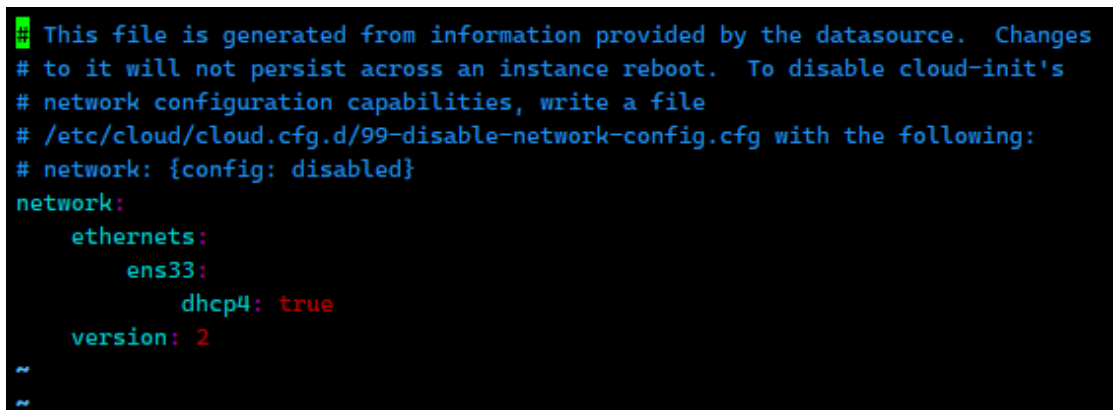
ubuntu24 修改 IP

默认安装好 /etc/netplan/ 目录下有自带的一个 50-cloud-init.yaml 文件，如果不想动原始文件，直接新建一个 01-netcfg.yaml 文件，然后写入下面配置，此方式需要你吧 50-cloud-init.yaml 文件中的网卡对应的 dhcp4 改成 false。

执行命令：sudo vim /etc/netplan/01-netcfg.yaml ， 根据自己需要固定的信息实际填写

动态 IP

vi /etc/netplan/50-cloud-init.yaml



固定 IP

vi /etc/netplan/01-netcfg.yaml

```
network:
  version: 2
  ethernets:
    ens33: #修改为你的实际接口名字
      dhcp4: no
      addresses:
        - 192.168.0.41/24 # 固定的 ip 地址
      routes:
        - to: default
          via: 192.168.0.1 # 网关
      nameservers:
        addresses:
          - 192.168.0.1 # dns1
          - 114.114.114.114 # dns2
```

地址：广州市 增城区 广深大道 邮编：510340
电邮：gzcovax@126.com QQ：3987797287

```
1 gitlab x +
network:
  version: 2
  ethernets:
    ens33: #修改为你的实际接口名字
      dhcp4: no
      addresses:
        - 192.168.0.41/24 # 固定的ip地址
      routes:
        - to: default
          via: 192.168.0.1 # 网关
      nameservers:
        addresses:
          - 192.168.0.1 # dns1
          - 114.114.114.114 # dns2
~
~
```

重启网络服务

```
sudo systemctl restart systemd-networkd
```

1.3.4 Gtlab 配置

修改配置文件位置 /etc/gitlab/gitlab.rb:

```
external_url 'http://192.168.0.41'
nginx['listen_port'] = 80
gitlab_rails['backup_path'] = "/var/opt/gitlab/backups"
gitlab_rails['backup_keep_time'] = 604800
```

```
##! On AWS EC2 instances, we also attempt to fetch the public hostname/IP
##! address from AWS. For more details, see:
##! https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instancedata-data-retrieval.html

external_url 'http://192.168.0.41'
nginx['listen_port'] = 8081
gitlab_rails['backup_path'] = "/var/opt/gitlab/backups"
gitlab_rails['backup_keep_time'] = 604800
```



配置文件位置 `/etc/gitlab/gitlab.rb`

```
[root@centos7 test]# vim /etc/gitlab/gitlab.rb
```

```
external_url 'http://192.168.0.41' # 这里一定要加上 http://
```

```
nginx['listen_port'] = 8081
```

```
# 配置邮件服务
```

```
gitlab_rails['smtp_enable'] = true
```

```
gitlab_rails['smtp_address'] = "smtp.qq.com"
```

```
gitlab_rails['smtp_port'] = 25
```

```
gitlab_rails['smtp_user_name'] = "hgzerowzh@qq.com" # 自己的 qq 邮箱账号
```

```
gitlab_rails['smtp_password'] = "xxx" # 开通 smtp 时返回的授权码
```

```
gitlab_rails['smtp_domain'] = "qq.com"
```

```
gitlab_rails['smtp_authentication'] = "login"
```

```
gitlab_rails['smtp_enable_starttls_auto'] = true
```

```
gitlab_rails['smtp_tls'] = false
```

```
gitlab_rails['gitlab_email_from'] = "hgzerowzh@qq.com" # 指定发送邮件的邮箱地址
```

```
user["git_user_email"] = "shit@qq.com" # 指定接收邮件的邮箱地址
```

修改好配置文件后，要使用 `gitlab-ctl reconfigure` 命令重载一下配置文件，否则不生效。

```
gitlab-ctl reconfigure # 重载配置文件
```

重置并启动 GitLab

```
gitlab-ctl restart
```



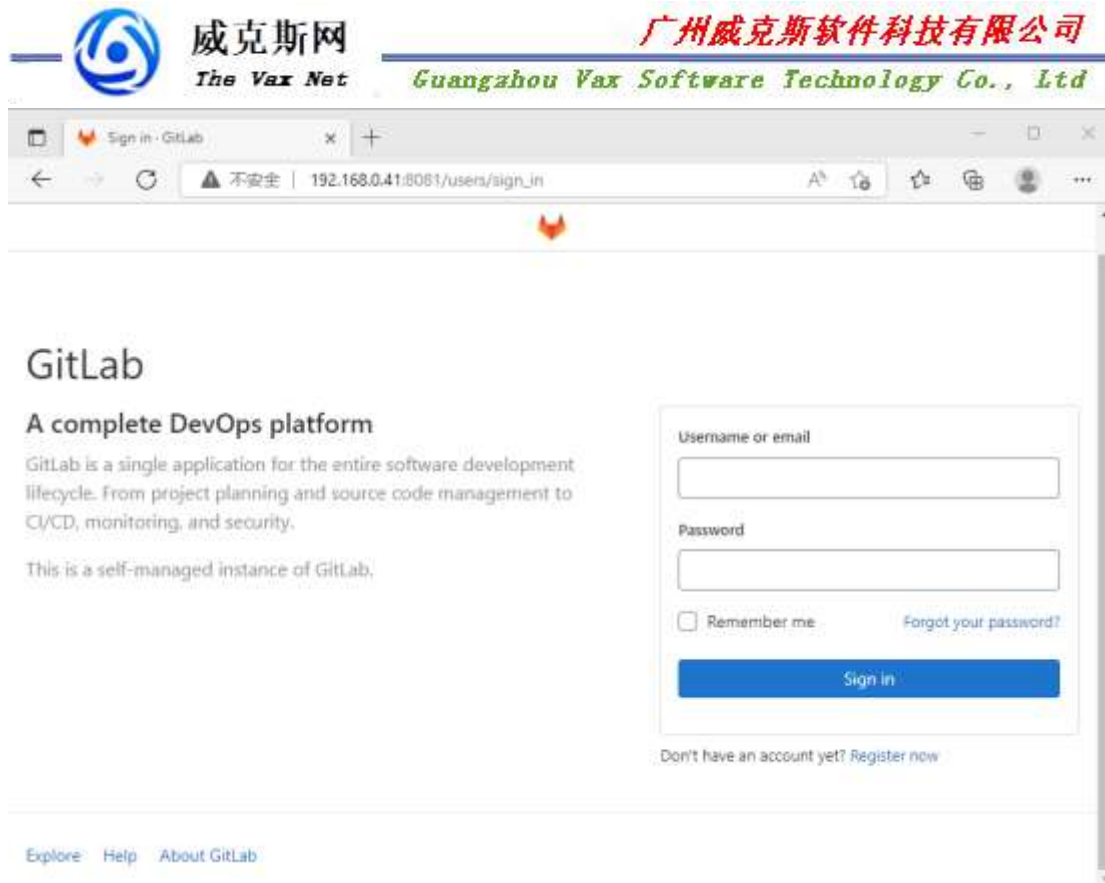

```
* link[/opt/gitlab/init/grafana] action create (up to date)
* file[/opt/gitlab/sv/grafana/down] action nothing (skipped due to action :nothing)
* directory[/opt/gitlab/service] action create (up to date)
* link[/opt/gitlab/service/grafana] action create (up to date)
* ruby_block[wait for grafana service socket] action run (skipped due to not_if)
(up to date)
Recipe: gitlab::database_reindexing_disable
* crond_job[database-reindexing] action delete
* file[/var/opt/gitlab/crond/database-reindexing] action delete (up to date)
(up to date)

Running handlers:
Running handlers complete
Chef Infra Client finished, 1/817 resources updated in 37 seconds
gitlab Reconfigured!
[root@server_nexus soft]# gitlab-ctl restart
ok: run: alertmanager: (pid 4332) ls
ok: run: gitaly: (pid 4340) 0s
ok: run: gitlab-exporter: (pid 4353) 0s
ok: run: gitlab-kas: (pid 4357) 0s
ok: run: gitlab-workhorse: (pid 4373) ls
ok: run: grafana: (pid 4381) 0s
ok: run: logrotate: (pid 4391) ls
ok: run: nginx: (pid 4397) 0s
ok: run: node-exporter: (pid 4408) ls
ok: run: postgres-exporter: (pid 4414) 0s
ok: run: postgresql: (pid 4464) ls
ok: run: prometheus: (pid 4515) 0s
ok: run: puma: (pid 4526) 0s
ok: run: redis: (pid 4531) 0s
ok: run: redis-exporter: (pid 4537) 0s
ok: run: sidekiq: (pid 4545) 0s
[root@server_nexus soft]#
```

5) 访问 GitLab 页面

输入地址:

<http://192.168.0.41:8081/>



1.3.5 用户密码

修改用户密码

获取/修改超级管理员 root 的密码

执行：`gitlab-rails console` 命令 开始初始化密码

在 `irb(main):001:0>` 后面通过 `u=User.where(id:1).first` 来查找与切换账号（`User.all` 可以查看所有用户）

通过 `u.password='12345678'` 设置密码为 12345678(这里的密码看自己喜欢):

通过 `u.password_confirmation='12345678'` 再次确认密码

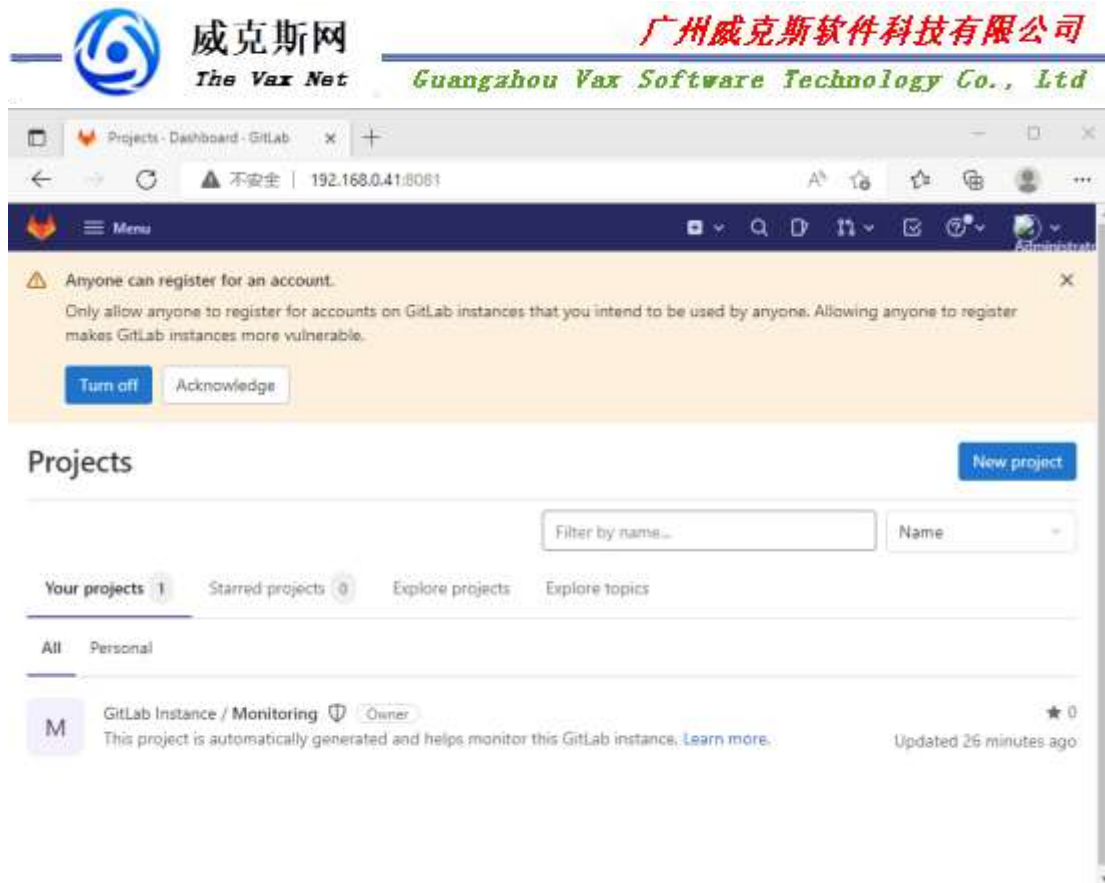
通过 `u.save!` 进行保存（切记切记 后面的！）

```
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]#
[root@server_nexus soft]# gitlab-rails console
-----
Ruby:      ruby 2.7.5p203 (2021-11-24 revision f69aeb8314) [x86_64-linux]
GitLab:    14.10.2 (07d12f3fd11) FOSS
GitLab Shell: 13.25.1
PostgreSQL: 12.7
-----[ booted in 27.56s ]
Loading production environment (Rails 6.1.4.7)
irb(main):001:0>
irb(main):002:0> u=User.where(id:1).first
=> #<User id:1 @root>
irb(main):003:0> u.password='aonst'
=> "aonst"
irb(main):004:0> u.password_confirmation='aonst'
=> "aonst"
irb(main):005:0> u.save!
=> true
irb(main):006:0> []
```

如果看到上面截图中的 **true** ，恭喜你已经成功了，执行 **exit** 退出当前设置流程即可。

回到 **gitlab** ,可以通过 **root/12345678** 这一超级管理员账号登录了

如果修改第二个用户密码: **u=User.where(id:2).first**



1.3.6 安装 Runner

安装方式一

下载适用于您系统的二进制文件

```
sudo curl -L --output /usr/local/bin/gitlab-runner https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-linux-amd64
```

授予其执行权限

```
sudo chmod +x /usr/local/bin/gitlab-runner
```

创建一个 GitLab Runner 用户

```
sudo useradd --comment 'GitLab Runner' --create-home gitlab-runner --shell /bin/bash
```

安装并作为服务运行

```
sudo gitlab-runner install --user=gitlab-runner --working-directory=/home/gitlab-runner
sudo gitlab-runner start
```

地址：广州市 增城区 广深大道
电邮：gzcovax@126.com

邮编：510340
QQ：3987797287

安装方式二

如果使用基于 `deb` 包的发行版

```
curl -s https://packages.gitlab.com/install/repositories/runner/gitlab-  
runner/script.deb.sh | sudo bash  
apt install -y gitlab-runner
```

如果使用基于 `rpm` 包的发行版

```
curl -s https://packages.gitlab.com/install/repositories/runner/gitlab-  
runner/script.rpm.sh | sudo bash  
yum install -y gitlab-runner
```

1.3.7 常用命令

```
gitlab-ctl start      # 启动所有 gitlab 组件  
gitlab-ctl stop       # 停止所有 gitlab 组件  
gitlab-ctl restart    # 重启所有 gitlab 组件  
gitlab-ctl status     # 查看服务状态
```

```
gitlab-ctl reconfigure # 启动服务  
gitlab-ctl show-config # 验证配置文件
```

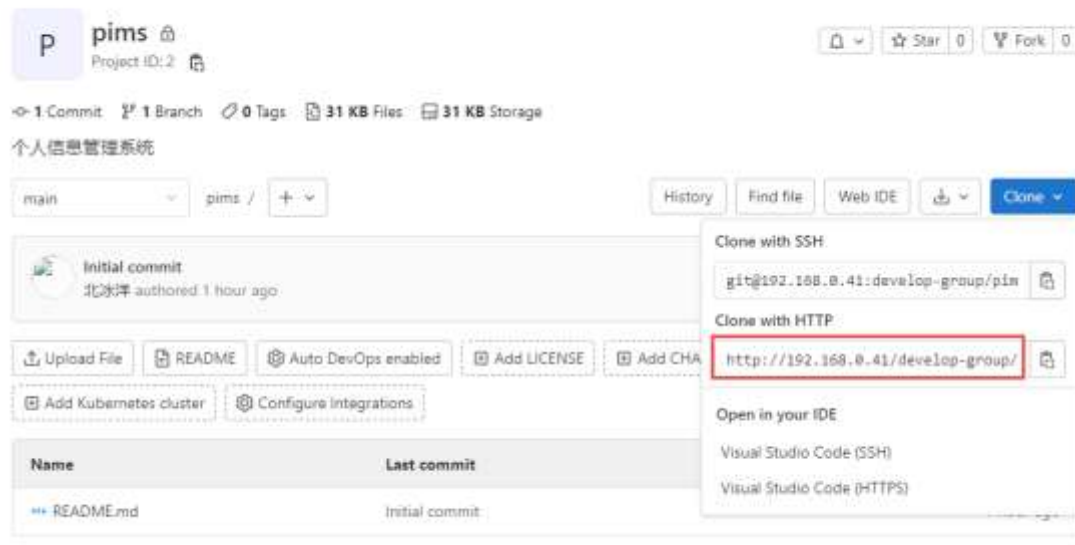
```
gitlab-ctl tail       # 查看日志
```

```
gitlab-rake gitlab:check SANITIZE=true --trace # 检查 gitlab
```

```
vim /etc/gitlab/gitlab.rb # 修改默认的配置文
```

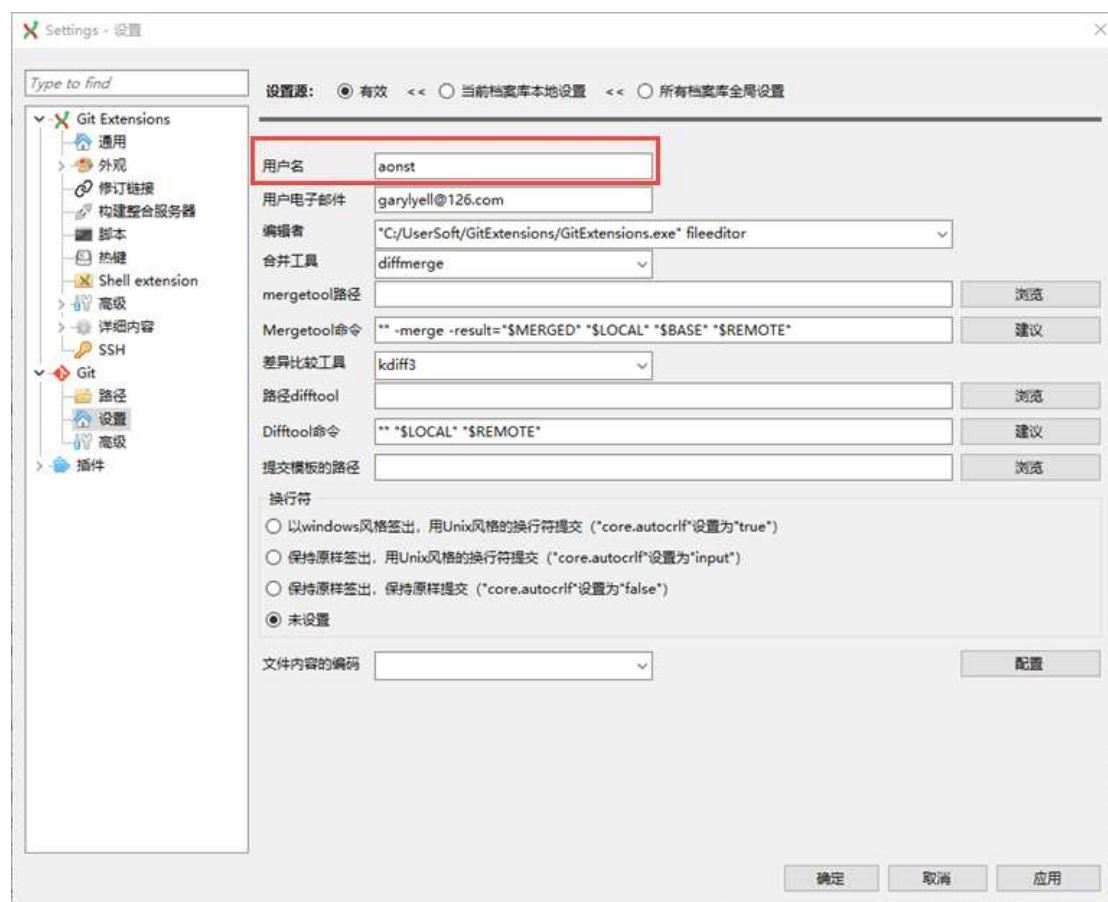
1.3.8 克隆项目

develop-group / pims



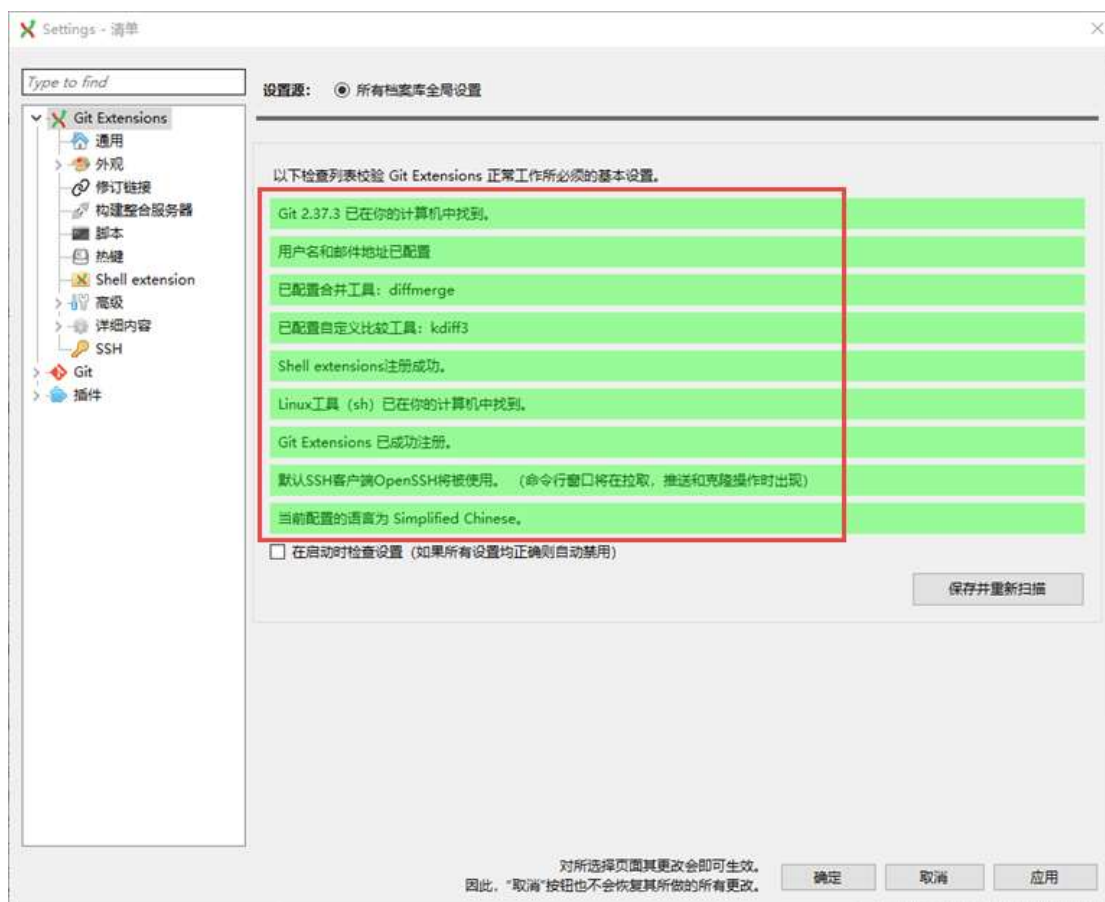
GitExtensions

安装客户端，设置用户信息



地址：广州市 增城区 广深大道
电邮：gzcorvax@126.com

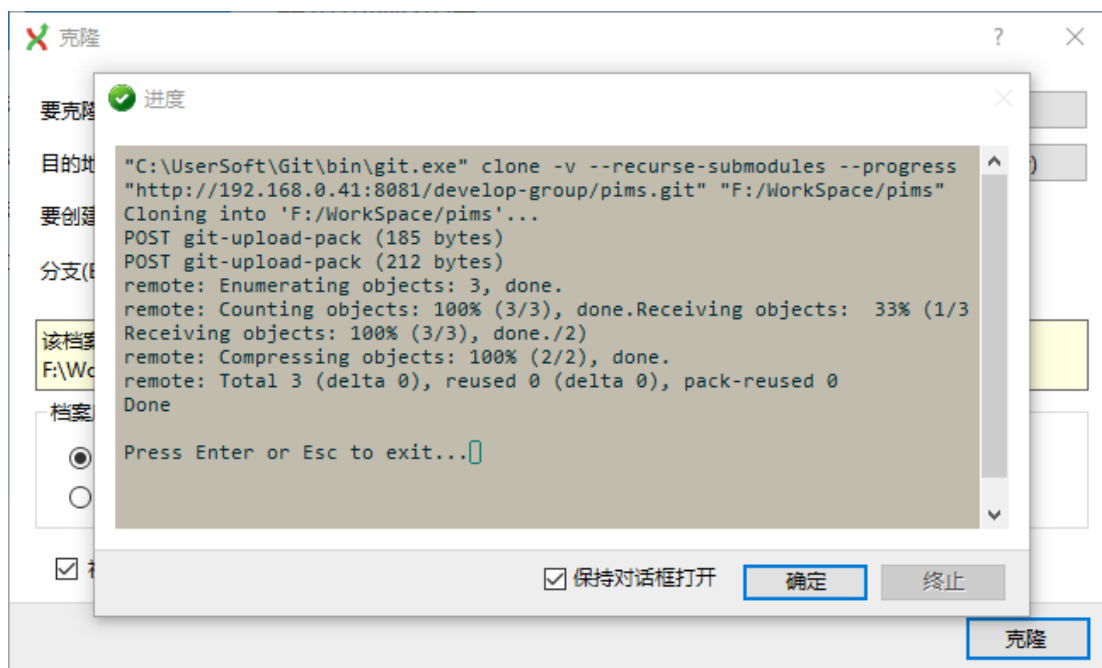
邮编：510340
QQ：3987797287



克隆项目: <http://192.168.0.41/develop-group/pims.git>

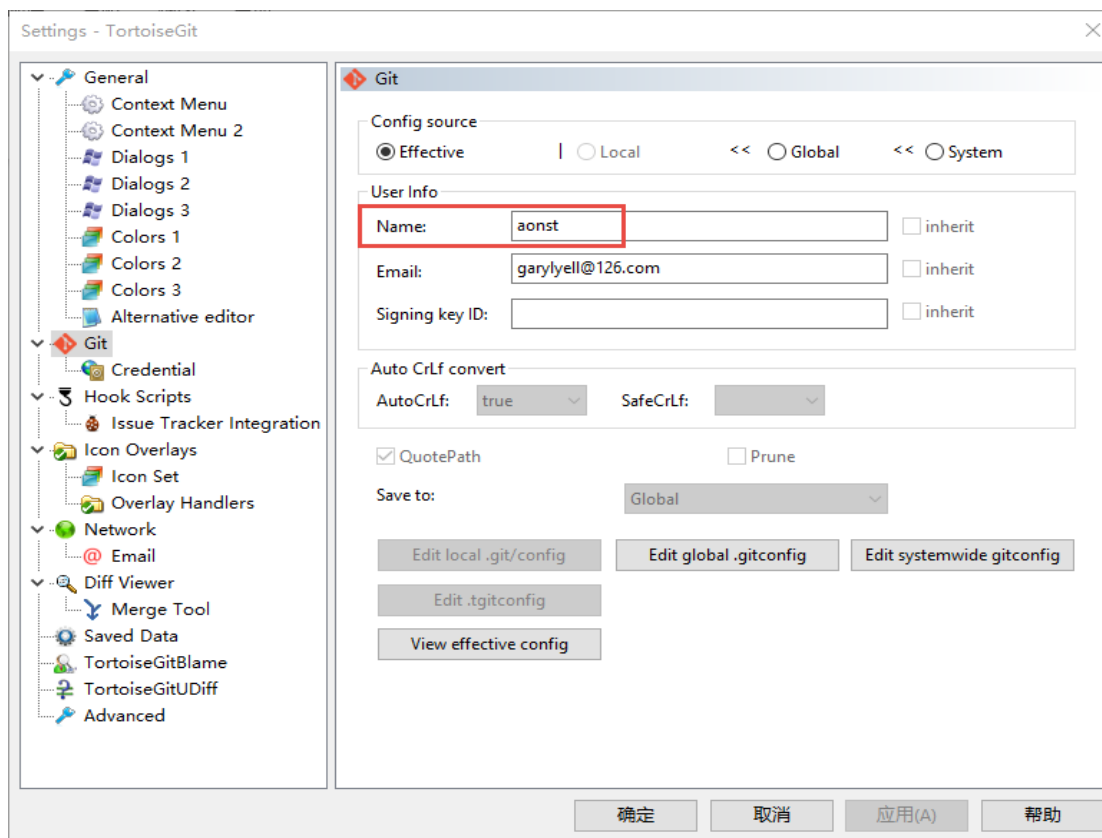
(端口不为 80 的话需要加上端口号)



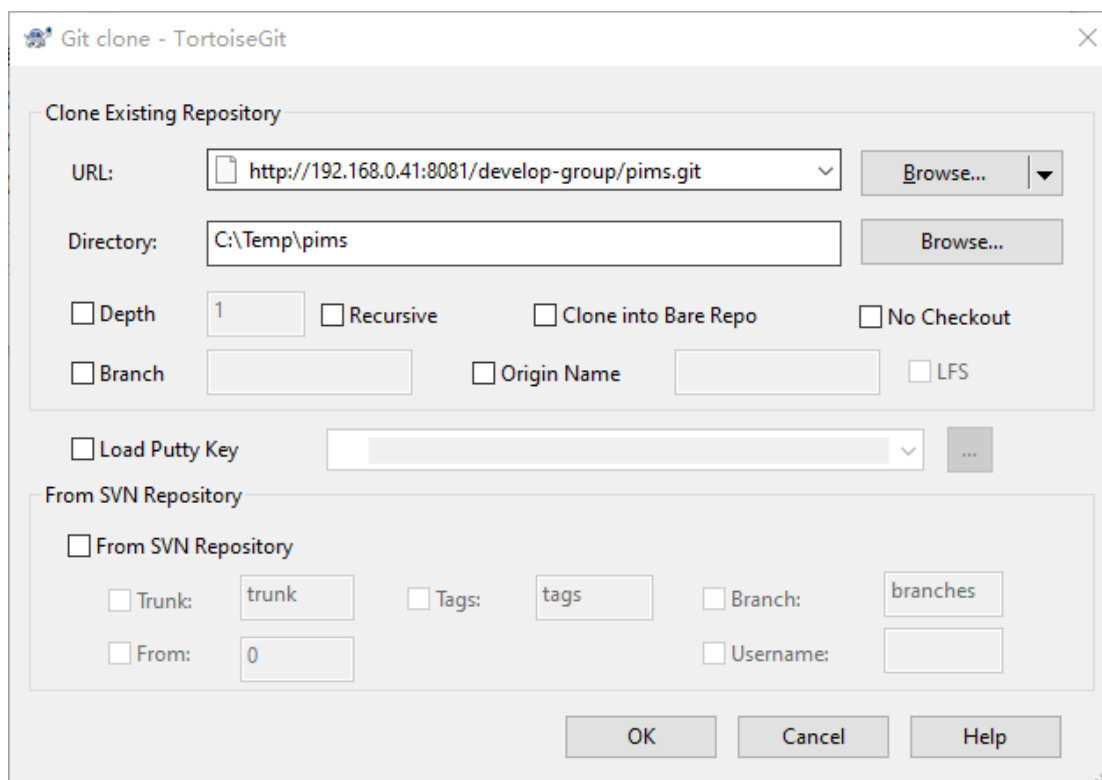


TortoiseGit

设置用户

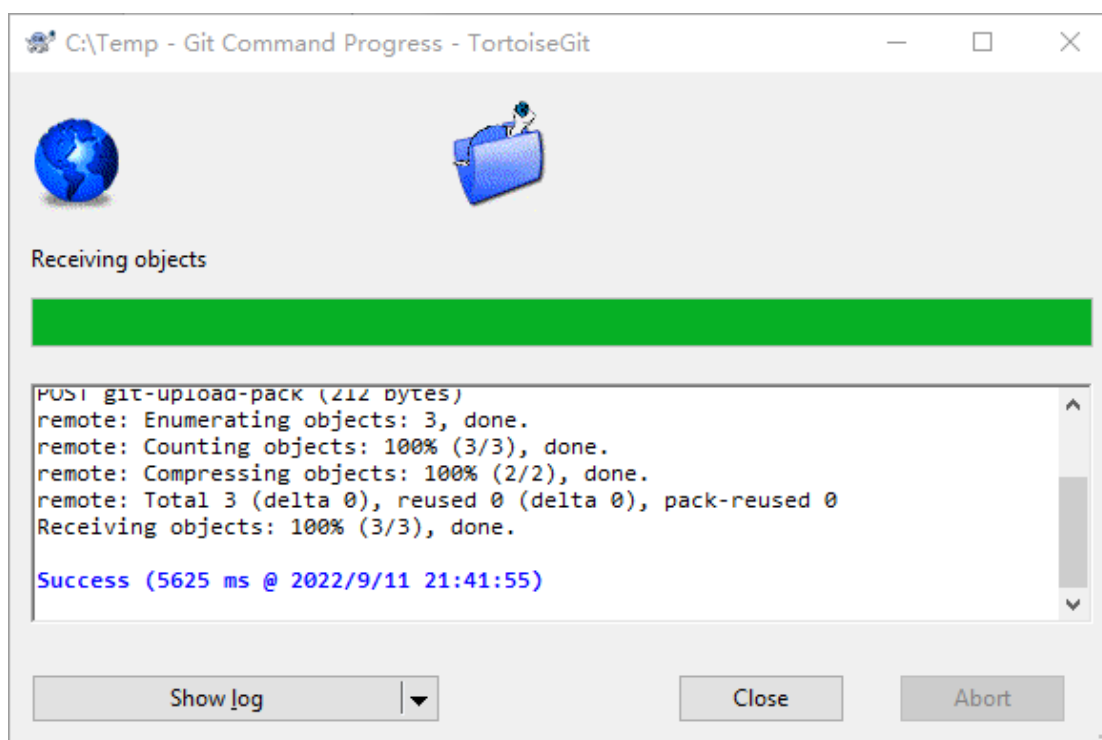


克隆:



地址: 广州市 增城区 广深大道
 电邮: gzcorvax@126.com

邮编: 510340
 QQ: 3987797287



1.4 使用说明

1.4.1 创建 Group

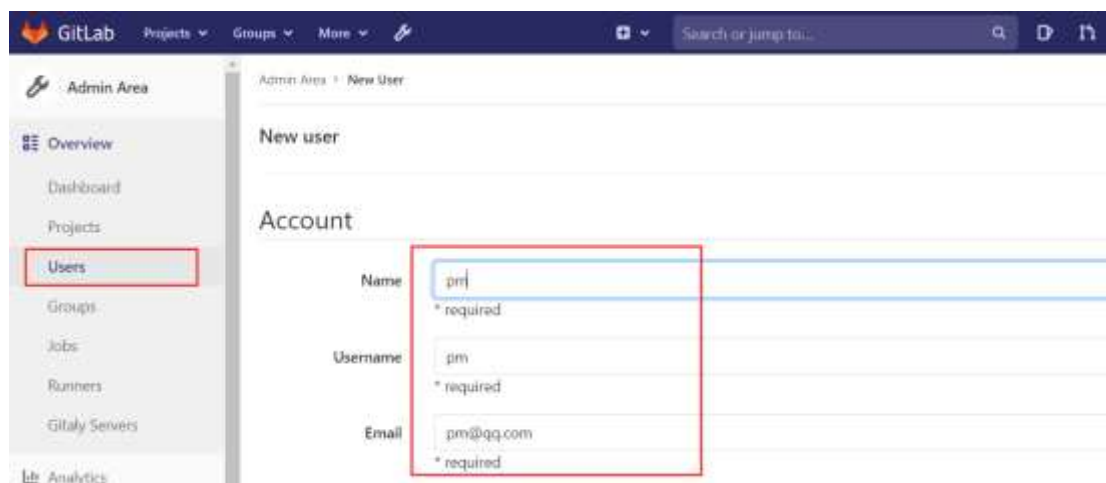
- 填上组名即可，这里组名为 java



1.4.2 创建 User

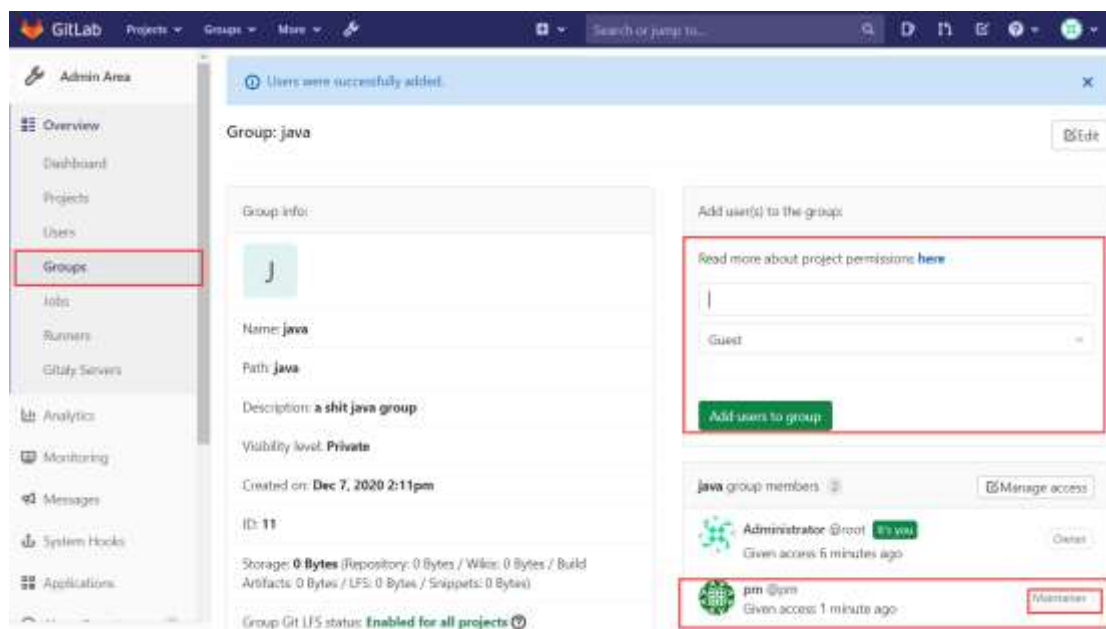
- 创建四个 User: pm、dev1、dev2、dev3





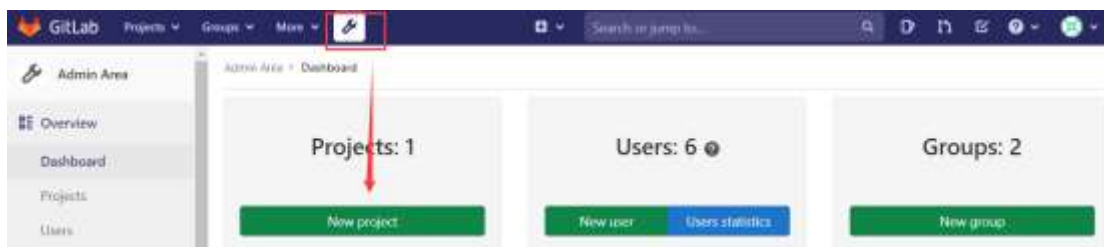
1.4.3 添加 User

添加 User 到 Group 中并授权



1.4.4 创建 Project

创建 Project 并配置 SSH



Blank project Create from template Import project

Project name
app01

Project URL Project slug
http://10.0.0.51/ java 属于哪个组 app01

Want to house several dependent projects under the same namespace? [Create a group.](#)

Project description (optional)
a shit java app

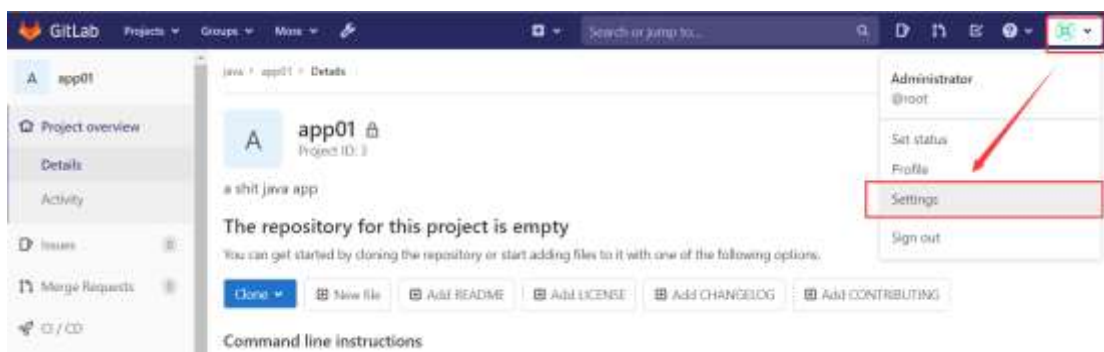
Visibility Level [?](#)

☒ Private
Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

☐ Internal

☐ Public

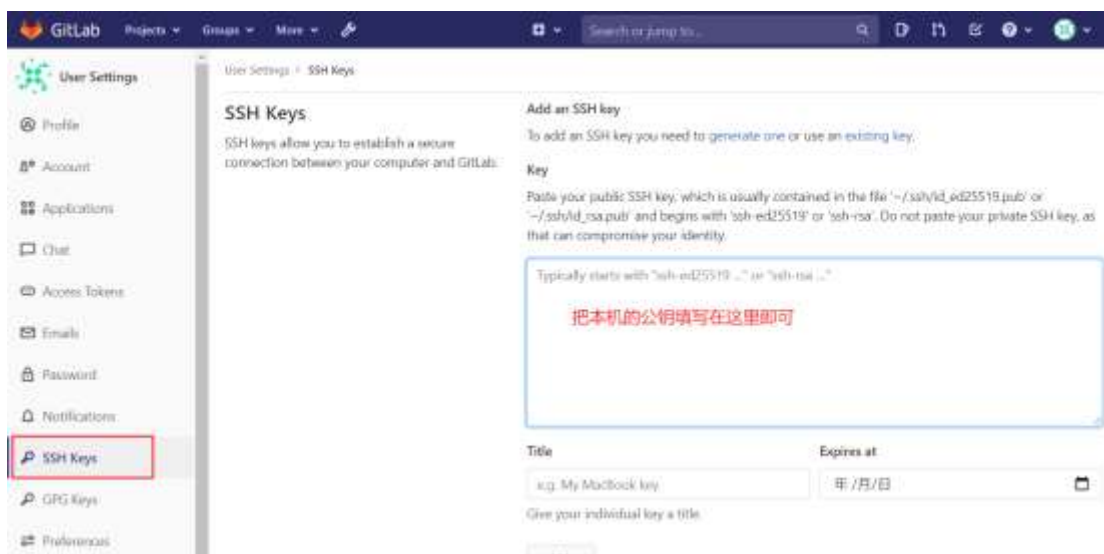
☐ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.



地

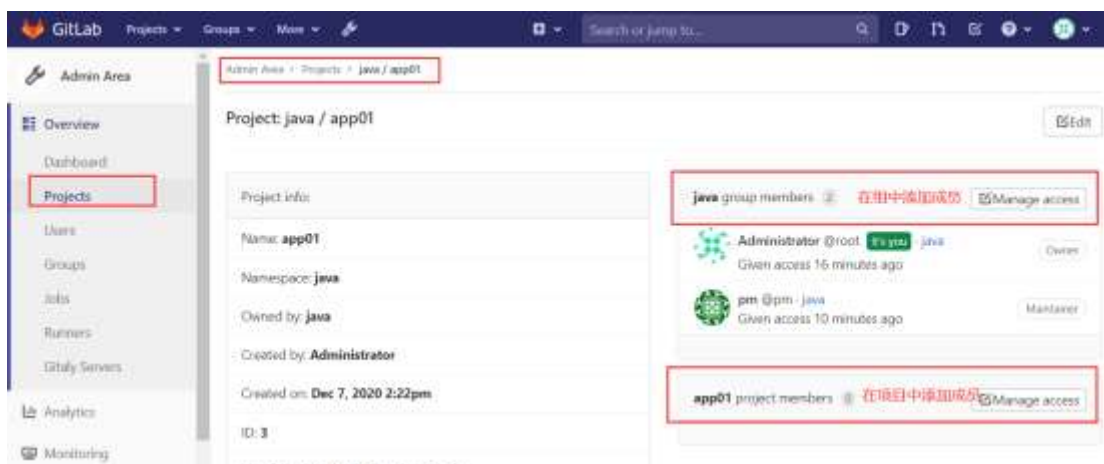
电邮: gzcovax@126.com

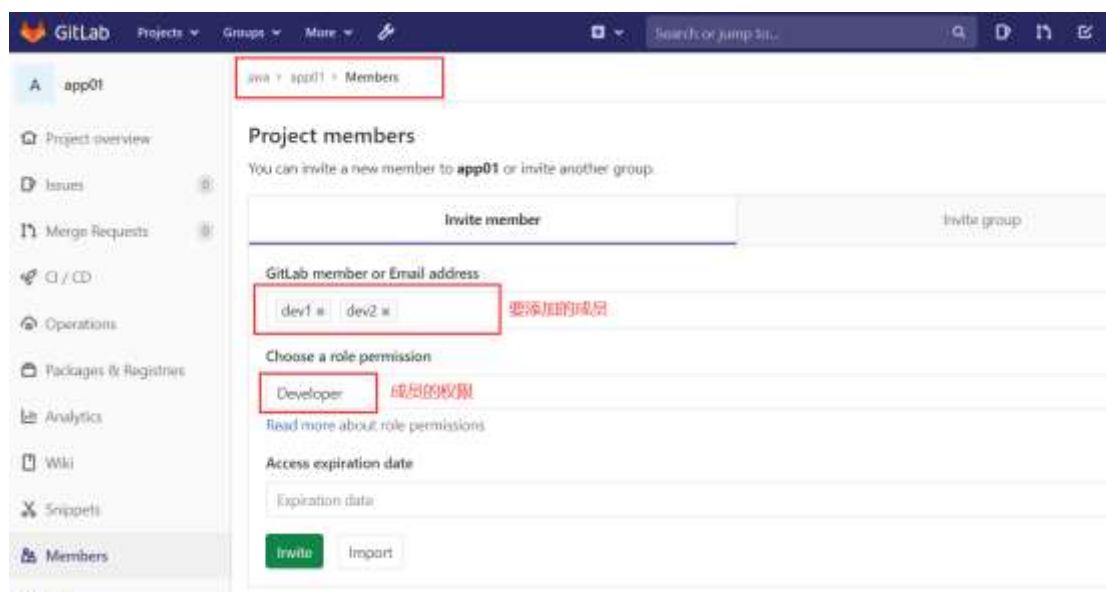
QQ: 3987797287



1.4.5 添加成员

在项目中添加成员





1.4.6 推送文件

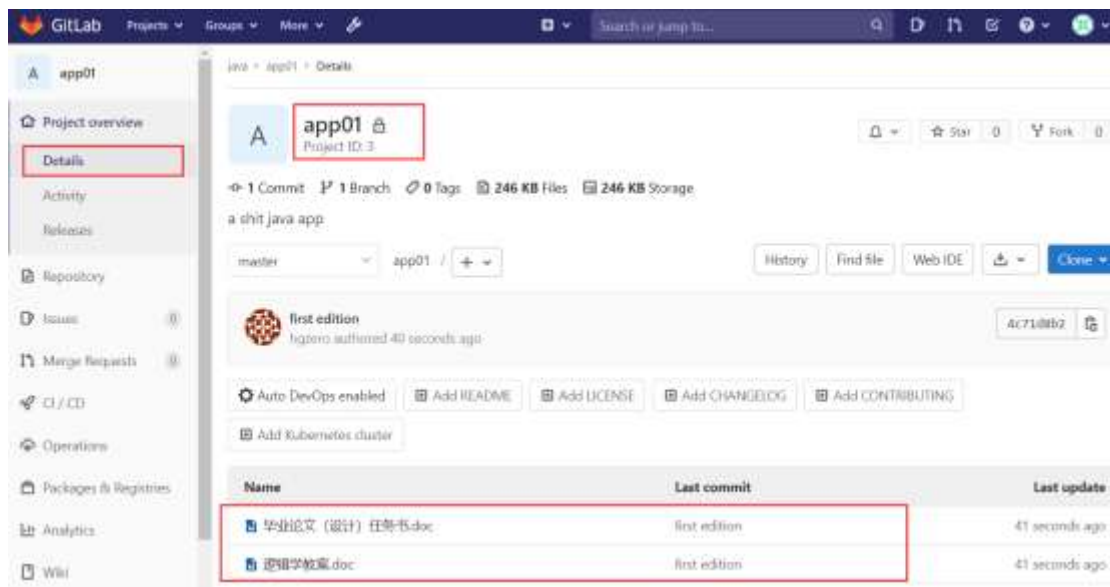
将本地文件推送到 Gitlab

```
# 将 app01 项目克隆下来
git clone git@10.0.0.51:java/app01.git

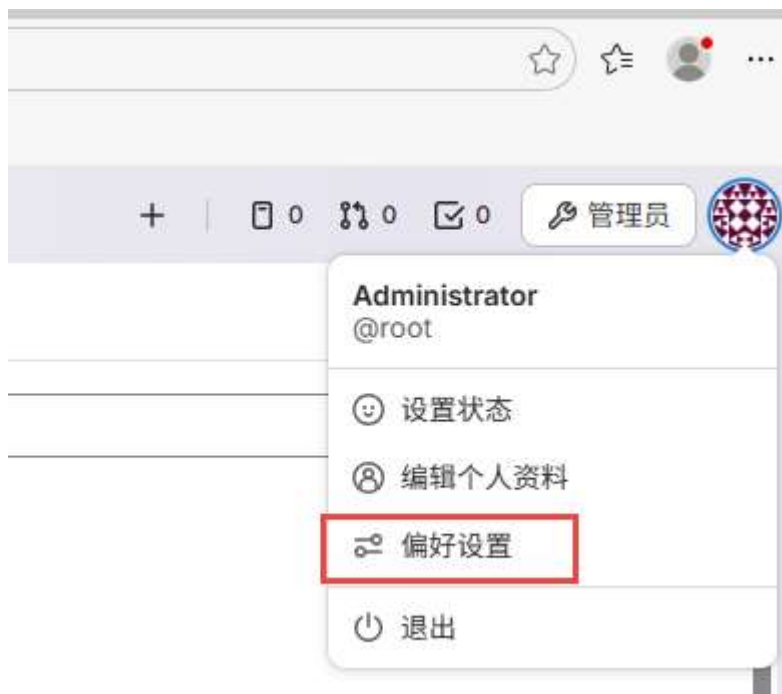
# 初始化配置
git config --global user.name "hgzero"
git config --global user.email "hgzero@qq.com"

# 在 app01 目录下新建一些文件

# 推送到 gitlab
git add .
git commit -m "first edition"
git push origin master
```

1.4.7 中文设置



用户设置

- ⑧ 用户资料
- 8* 帐户
- ⌘ 应用
- ✕ Integration accounts
- 💬 个人访问令牌
- ✉ 电子邮件
- 🔒 密码
- 🔔 通知
- 🔑 SSH Keys
- 🔑 GPG 密钥
- 🔗 偏好设置
- 💬 评论模板
- 🖨 Active sessions
- 📅 认证日志

个性化

本地化

自定义语言和区域相关设置。 [进一步了解](#).

语言

Chinese, Simplified - 简体中文 (82% 已翻译)

此功能是实验性的，翻译尚未完成。
[帮助将 GitLab 翻译成您的语言](#) ↗

每周的起始日

系统默认值 (星期日)

1.4.8 调整路径

调整项目路径

进入项目设置

打开要迁移的项目

左侧菜单 → Settings → General

找到 Advanced 部分，点击展开

执行迁移

在 Advanced settings 中找到 Transfer project 部分

点击 Transfer project 按钮

在 Select a new namespace 下拉框中，选择目标组

输入项目名称以确认

点击 Transfer project 确认

地址：广州市 增城区 广深大道

电邮：gzcovax@126.com

邮编：510340

QQ：3987797287

项目

hist / pims-est

在您的个人资料中添加SSH密钥之前，您不能通过SSH来拉取或推送仓库。

添加SSH密钥 不再显示

pims-est

该项目仓库当前为空
要开始使用，请克隆仓库或上传一些文件。

命令行指引

您还可以按照以下说明从计算机中上传现有文件。

配置您的 Git 身份

开始使用 Git 并学习 如何配置它。

本地 全局

Git 本地设置

设置

通用

集成

Webhooks

访问令牌

仓库

添加项目描述



Q 搜索页

命名、描述、主题

更新您的项目名称、描述、头像和主题。了解更多关于项目的信息。

项目名称

pims-est

项目ID

1

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.



项目头像

选择文件...

未选定任何文件。

理想的图像尺寸为 192 x 192 像素。允许的最大文件大小为 200 KiB。

项目描述 (可选)

个人信息管理系统 (JDK1.8版本)

项目主题

搜索主题

No matches found

不要在主题名称中包含敏感信息。了解更多。

保存更改

迁移项目

- 您需要更新本地仓库以指向新位置。

路径

http://192.168.0.41:8081/hist/ pims-est

更改路径

转移项目

将您的项目转移到另一个命名空间。 [了解更多](#)。

一个项目的仓库名称定义了它的URL（用来通过浏览器访问项目）以及它在安装GitLab 的文件磁盘上的位置。

当您项目转移到群组时，您可以轻松管理多个项目，查看存储、计算分钟和用户的使用配额，并开始试用或：

没有群组？ [创建一个](#)

转移前需要注意的事项：

- 请注意，更改项目的命名空间可能会产生非预期的副作用。
- 您只能将项目转移到您管理的命名空间。
- 您需要更新本地仓库以指向新位置。
- 当项目转移到群组后，其可见性级别将更改为与命名空间规则匹配。

选择一个新的命名空间

hist / pims

转移项目

1.4.9 禁止流水线

禁止项目所有流水线运行

完全禁止项目所有流水线运行

这通常在项目初始化、维护或归档时使用。

操作方法：在项目设置中关闭 CI/CD。

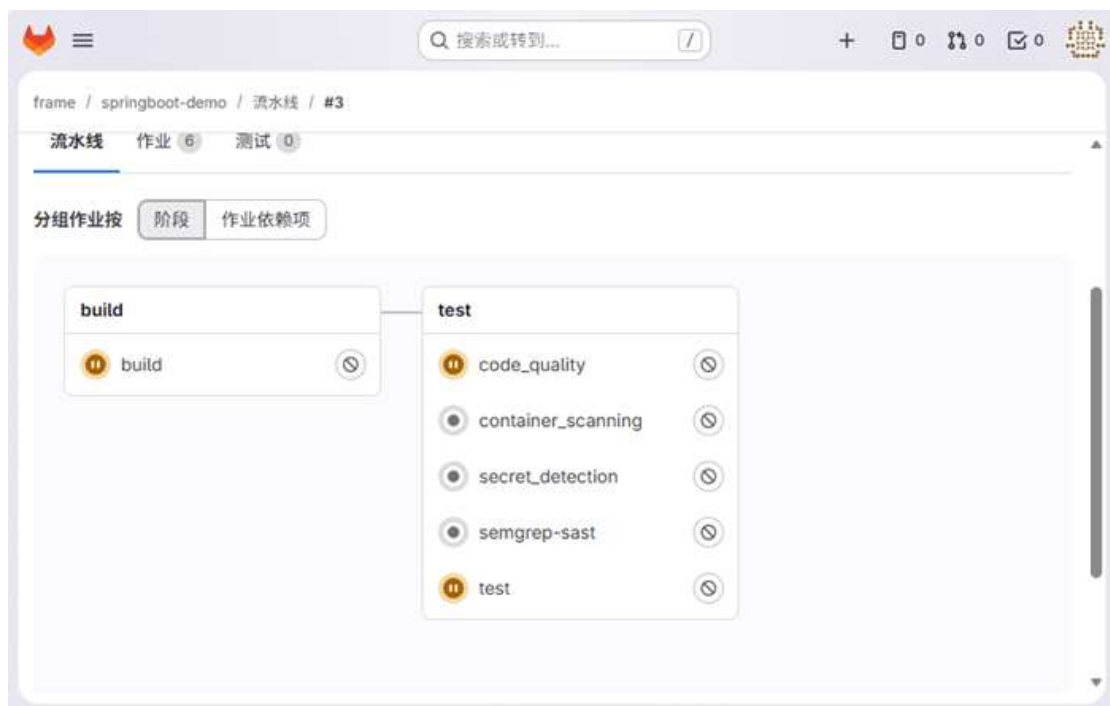
进入项目 Settings → General。

展开 “Visibility, project features, permissions” 区域。

找到 “CI/CD” 选项，将开关切换为 “Disabled”。

点击 “Save changes”。

效果：此后，任何推送、合并请求或手动操作都不会触发任何 CI/CD 流水线。`.gitlab-ci.yml` 文件将被忽略。



1.4.10 创建和注册 Runner

地址: 广州市 增城区 广深大道
电邮: gzcovax@126.com

邮编: 510340
QQ: 3987797287

项目

frame / springboot-demo

S springboot-demo

已钉选
议题
合并请求

管理
计划
代码
构建
安全
部署
运维
监控
分析
设置

通用
集成
Webhooks
访问令牌
仓库
合并请求
CI/CD
软件包与镜像库

在您的个人资料中添加SSH密钥之前，您不能通过SSH来拉取或推送仓库。
添加SSH密钥 不再显示

S springboot-demo

main springboot-demo

initial
aonst authored 13小时前

名称	最后提交
src	initial
.gitignore	initial
README.md	initial
pom.xml	initial

README.md

springboot-demo

frame / springboot-demo / CI/CD 设置

流水线通用设置
自定义您的流水线配置。

Auto DevOps
根据持续的集成和交付配置，自动构建，测试和部署您的应用程序。我该如何开始？

Runner
Runner用于接收和执行GitLab的CI/CD作业的进程。什么是 Runner？

可用的 Runner

创建项目 Runner

分配的项目 runner Other available project runners 1 群组 实例



Runner 已创建。

注册 runner

平台

操作系统

☒ Linux ☐ macOS ☐ Windows

容器

☒ Docker ☐ Kubernetes

必须先安装 Runner 才能注册 runner。如何安装 Runner?

步骤 1

复制并粘贴以下命令到您的命令行中，注册 runner。

```
$ gitlab-runner register  
--url http://192.168.0.41  
--token glrt-537AtfvYo8PEtIrcCQTROW86MQpw0jkKdDozCnU6Mg8.01.170476hxx
```

① Runner 身份验证令牌 `glrt-537AtfvYo8PEtIrcCQTROW86MQpw0jkKdDozCnU6Mg8.01.170476hxx` 在此处仅短暂显示。在您注册 Runner 后，此令牌将失效。

步骤 2

当命令行提示时，选择一个执行器。执行器在不同的环境中运行构建。不能确定选择哪一个？

```
root@glserver:~#  
root@glserver:~# gitlab-runner register --url http://192.168.0.41 --token glrt-537AtfvYo8PEtIrcCQTROW86MQpw0  
jkKdDozCnU6Mg8.01.170476hxx  
Runtime platform arch=amd64 os=linux pid=13881 revision=9ffb4aa0 version=18  
.8.0  
Running in system-mode.  
  
Enter the GitLab instance URL (for example, https://gitlab.com/):  
[http://192.168.0.41]:  
Verifying runner... is valid correlation_id=01KH2G3GVZM251D0HREQZGPWY2 runner=537AtfvYo  
Enter a name for the runner. This is stored only in the local config.toml file:  
[glserver]:  
Enter an executor: ssh, docker, docker+machine, kubernetes, instance, custom, parallels, virtualbox, docker-wi  
ndows, docker-autoscaler, shell:  
shell  
Runner registered successfully. Feel free to start it, but if it's running already the config should be automa  
tically reloaded!  
  
Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"  
root@glserver:~#
```

步骤 3 (可选)

手动验证 runner 是否可以选中作业。

```
$ gitlab-runner run
```

如果您将 runner 作为 系统或用户服务 [🔗](#)，则可能不需要。

您已注册了一个新的运行器！

您的运行器在线并准备好运行作业。

要查看 runner，请进入 [项目](#) > [CI/CD 设置](#) > [Runners](#)。

[查看运行器](#)

Runner

Runner用于接收和执行GitLab的CI/CD作业的进程。 [什么是 Runner?](#)

可用的 Runner	
分配的项目 runner 1	Other available project runners 1 群组 实例
状态	运行器配置 ?
● 在线 ○ 空闲	#3 (537AtfvY) 📁 项目 👤 0 🕒 最后联系: 1分钟前 📄 192.168.0.41 👤 由 aonst 创建于 10分钟前 springboot

[查看作业执行情况](#)

Runner #3 (537AttvY)

● 在线

📄 项目

由 aonst 创建于 34分钟前

描述

无

最后联系

4分钟前

配置

运行未打标签的作业

最大作业超时

10分

令牌过期

永不过期

标签

无

已分配的项目 1

Q 过滤项目

S

frame / springboot-demo

所有者

SpringBoot框架模板

Runner 1

系统 ID	状态	版本	IP地址	执行器	架构/平台
s_daa6c7087b2e	● 在线	18.8.0 (9ffb4aa0)	192.168.0.41	shell	amd64/linux

作业 3

状态	作业	项目	提交	完成于	时长	队列中
❌ 已失败	#48	springboot-demo	80881c82	4分钟前		00:05:20
❌ 已失败	#47	springboot-demo	80881c82	4分钟前	00:00:02	00:05:19
⚠️ 已失败	#46	springboot-demo	80881c82	4分钟前	00:00:02	00:18:30

1.5 备份还原

1.5.1 备份 gitlab

1) 修改配置文件

- `/etc/gitlab/gitlab.rb`

备份保存的位置，这里是默认位置，可修改成指定的位置

```
gitlab_rails['backup_path'] = "/var/opt/gitlab/backups"
```

设置备份保存的时间，超过此时间的日志将会被新覆盖

```
gitlab_rails['backup_keep_time'] = 604800 # 这里是默认设置，保存 7 天
```

特别注意：

如果自定义了备份保存位置，则要修改备份目录的权限，比如：

```
# chown -R git.git /data/backup/gitlab
```

地址：广州市 增城区 广深大道 邮编：510340
电邮：gzcovax@126.com QQ： 3987797287

- 配置完成后要重启以使配置生效

重读配置文件

```
gitlab-ctl reconfigure
```

重启 gitlab

```
gitlab-ctl restart
```

2) 设置定时任务

设置编辑器: `select-editor`

```
root@glserver:~# select-editor

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 2
```

每天凌晨 2 点定时创建备份

将以下内容写入到定时任务中 `crontab -e`

```
0 2 * * * /usr/bin/gitlab-rake gitlab:backup:create
```

备份策略建议:

本地保留 3 到 7 天, 在异地备份永久保存

每周日 21 点定时执行备份 (0 和 7 都代表周日)

```
0 21 * * 0 /root/backup.sh
```

```
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 21 * * 7 /root/backup.sh
```

记录日志

```
0 21 * * 7 /root/backup.sh >> /root/backup.log 2>&1
```

地址: 广州市 增城区 广深大道 邮编: 510340
电邮: gzcovax@126.com QQ: 3987797287

确认 cron 服务运行

```
bash
```

```
systemctl status cron # 或 crond (不同发行版可能不同)
```

如果未运行，启动并设置开机自启：

```
bash
```

```
sudo systemctl enable --now cron
```

验证任务是否生效

编辑完成后保存退出，可以列出当前用户的定时任务：

```
bash
```

```
crontab -l
```

应该就能看到刚刚添加的那一行。

这样，每周日晚上 9 点，系统就会自动执行 backup.sh 了。

3) 备份时间的识别

备份后的文件类似这样的形式：1494170842_gitlab_backup.tar，可以根据前面的时间戳确认备份生成的时间

```
data -d @1494170842
```

1.5.2 恢复 gitlab

1) 停止数据写入服务

停止数据写入服务

```
gitlab-ctl stop unicorn
```

```
gitlab-ctl stop sidekiq
```

2) 进行数据恢复并重启

进行恢复

```
gitlab-rake gitlab:backup:restore BACKUP=1494170842 # 这个时间戳就是刚刚备份的文件前面的时间戳
```

```
gitlab-rake gitlab:backup:restore BACKUP=1769470224_2026_01_27_14.10.2
```

地址：广州市 增城区 广深大道

邮编：510340

电邮：gzcovax@126.com

QQ：3987797287

gitlab			
/var/opt/gitlab/backups			
名称	大小	类型	修改时间
..			
1769470224_2026_01_27_14.10.2_gitlab_backup.tar	245.32MB	压缩存档...	2026/2/9, 5:44

请确保 GitLab 版本一致

GitLab version mismatch:

Your current GitLab version (18.8.3) differs from the GitLab version in the backup!

Please switch to the following version and try again:

version: 14.10.2

重启

gitlab-ctl restart

1.5.3 批量导出项目

进入管理员后台

点击顶部菜单的 Admin Area（管理员区域）。

选择项目批量导出

左侧菜单依次进入：

Settings → General → Visibility and access controls

找到 Project export 部分。

开启并执行导出

勾选 "Enable project export"（通常默认开启）

点击 "Export all projects"（导出所有项目）

获取导出文件

系统会在后台生成导出任务，完成后：

管理员邮箱会收到下载链接

地址：广州市 增城区 广深大道

邮编：510340

电邮：gzcorvax@126.com

QQ：3987797287

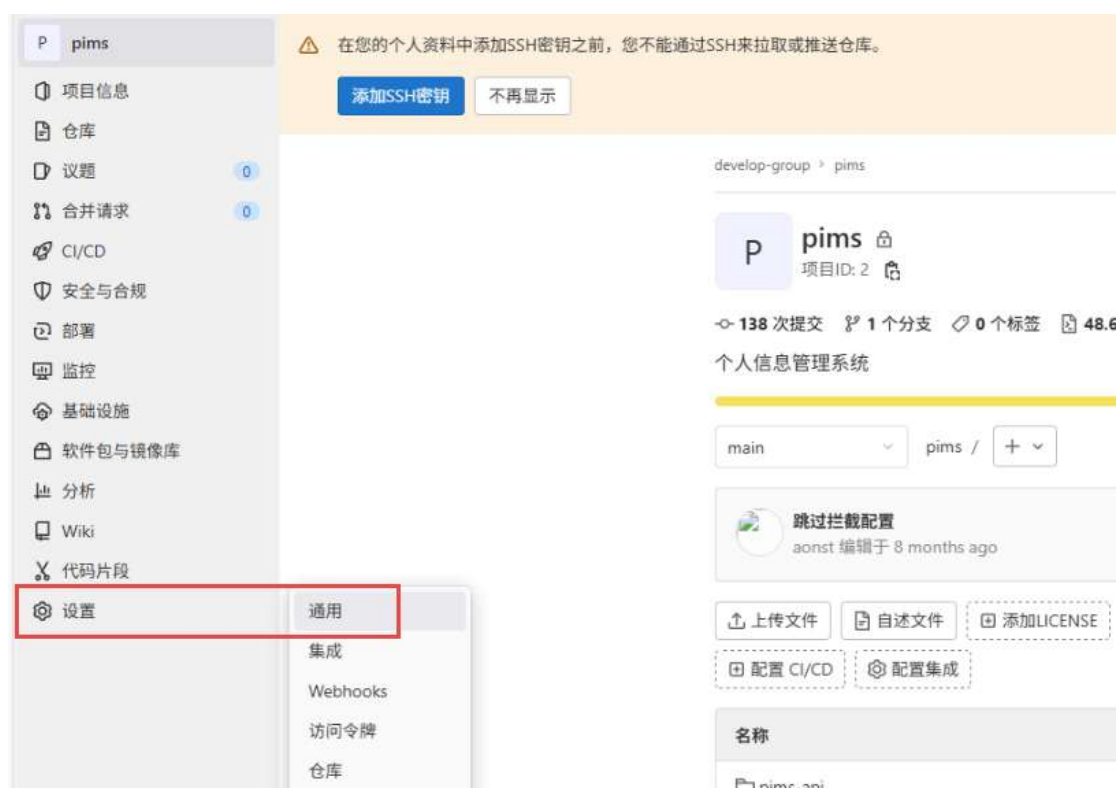
或在 Admin Area → Projects → 点击项目 → Exports 页面下载

GitLab 14.7 及以上 在管理员后台增加了更直观的批量导出口：

Admin Area → Settings → General → Project export

可直接勾选多个项目并批量导出。

1.5.4 导出项目





高级

例行维护、导出、归档、更改路径、传输和删除。

例行维护

在当前仓库中运行一些例行维护任务，例如压缩文件修订和删除无法访问的对象。 [了解更多。](#)

运行例行维护

导出项目

导出此项目及其所有相关数据，以便将其移动到新的 GitLab 实例。导出的文件准备就绪后，您可以从此页面或从您将收到的电子邮件通知中的下载链接下载它。然后，您可以在创建新项目时将其导入。 [了解更多。](#)

以下项目将会被导出：

- 项目和wiki仓库
- 项目上传
- 项目配置，不包括集成
- 议题评论，合并请求的差异和评论，标记，里程碑，代码片段和其他项目实体
- LFS 对象
- 议题看板
- 设计管理文件和数据

以下项目将不会被导出：

- 作业日志和产物
- 容器镜像库镜像
- CI 变量
- 流水线触发器
- Webhooks
- 任何加密的令牌

导出项目

 项目导出已经开始。下载链接将通过电子邮件发送并在此页面上提供。

等待项目导出完成，下载导出：

高级

例行维护、导出、归档、更改路径、传输和删除。

例行维护

在当前仓库中运行一些例行维护任务，例如压缩文件修订和删除无法访问的对象。[了解更多](#)。

运行例行维护

导出项目

导出此项目及其所有相关数据，以便将其移动到新的 GitLab 实例。导出的文件准备就绪后，您可以从此页面或从您将收到的电子邮件通知中的下载链接下载它。然后，您可以在创建新项目时将其导入。[了解更多](#)。

以下项目将会被导出：

- 项目和wiki仓库
- 项目上传
- 项目配置，不包括集成
- 议题评论，合并请求的差异和评论，标记，里程碑，代码片段和其他项目实体
- LFS 对象
- 议题看板
- 设计管理文件和数据

以下项目将不会被导出：

- 作业日志和产物
- 容器镜像库镜像
- CI 变量
- 流水线触发器
- Webhooks
- 任何加密的令牌

下载导出

生成新的导出

1.5.5 导入项目

使用管理员账号登陆，设置导入导出

管理区域

- 概览
- CI/CD
- 分析
- 监控
- 消息
- 系统钩子
- 应用程序
- 举报滥用
- 部署密钥
- 标记
- 设置
- 通用
- 搜索
- 集成
- 仓库

管理区 / 通用

导入和导出设置

配置导入源以及导入导出功能相关的设置。

导入源
代码可在项目创建期间从已启用的源导入。必须为 GitHub?

- ☒ GitHub
- ☐ Bitbucket Cloud
- ☐ Bitbucket Server
- ☐ FogBugz
- ☐ Repository by URL
- ☒ GitLab export
- ☐ Gitea
- ☐ Manifest file

导出项目
☒ 已启用

允许通过直接传输迁移群组 and 项目
☐ 已启用



创建空白项目

创建一个空白项目来存放您的文件，规划您的工作，并在代码等方面进行协作。



从模板创建

创建一个预先填充了...



导入项目

从外部源（如 GitHub、Bitbucket 或 GitLab 的其他实例）迁移数据。

导入项目

从外部源（如 GitHub、Bitbucket 或 GitLab 的其他实例）迁移数据。

导入项目自

💡 Migrating GitLab projects? Migrating projects when you migrate gro

 GitLab导出

 GitHub

GitLab导入

导入一个从GitLab导出的项目

项目名称

java-case

项目 URL

http://192.168.0.41/ case

项目标识串

java-case

如需将整个GitLab项目从另一个GitLab服务器移动或复制到此服务器，请访问原项目的设置页面，生成导出文件，然后在此

GitLab项目导出

选择文件 2026-02-12_12-...de_export.tar.gz

导入项目

取消

该项目已成功导入。

J

java-case

main

java-case

+

查找文件

代码

:

 案例分类

aonst authored 2024年6月10日

e1f0312c



历史

名称	最后提交	最后更新
src	案例分类	1年前
.gitignore	添加案例	1年前
README.md	Initial commit	1年前
case.docx	添加案例	1年前

README.md

1.6 参考文件